Application of Machine Learning to Tennis Shot Quality Analysis

Shiven Mehta
Saint Francis High School, 1885 Miramonte Ave, Sunnyvale CA 94040, USA:
shiven.mehta12@gmail.com

ABSTRACT

One of the most effective ways for players to improve their tennis performance is by analyzing match footage to identify shots that are poorly executed or fail to achieve the intended impact on the opponent. However, reviewing match footage and documenting each point is extremely time consuming, especially for matches that last hours. When estimating the quality of their shots, players have to manually estimate factors like speed, landing position, and impact on opponents. Having an AI driven model perform this process can efficiently automate this analysis and uncover patterns in a player's game. This paper explores an AI approach to scoring tennis shot quality for individual shots on a continuous scale from zero to ten, utilizing factors such as depth, width, player positions, and speed. It extracts data from two deep convolutional neural networks (TrackNet and Faster R-CNN), a categorical gradient boosting model (CatBoostRegressor), and a mathematical binary moving average model. By training a highly efficient gradient boosting model (Light GBM) on these extracted features, a mean squared error of 0.27, a mean absolute percentage error of 7.31%, and a mean absolute error of 0.44 was achieved when classifying shot quality for players on both sides of the court.

INTRODUCTION

Tennis matches can last multiple hours. Many players then spend hours analyzing their matchplay following a match, writing notes down in a notebook. Each shot is critical in tennis, so documenting what happens in each point shot by shot is critical for players trying to improve their tennis game. Analyzing match play allows players to understand the areas of their game they need to work on. This allows them to play better for following matches and improve certain aspects of their game such as movement, specific shots, and placement of shots. For example, if a player notices that their forehand is consistently falling short into the court, allowing other players to attack the shot, the player can work on hitting deeper forehands in practice. Tennis shot quality analysis is crucial for expediting and improving post-match evaluations, benefiting not only amateur players but also professionals aiming to better their performance. Limited research has been conducted in the field of tennis shot analysis, making this study especially important for those looking to further explore and expand upon the existing work. A model that can take a RGB video of a tennis match to assess shot quality could be significantly improved by incorporating shot classification. Each type of shot employs distinct techniques, and much of the existing research in tennis focuses on these technical aspects.

Current research on machine learning applications to tennis match analysis consists of advancement in human activity recognition and focuses on shot classification. Similar research in shot classification and machine learning based sports analysis is significant in sports, such as basketball and hockey.

The ATP Tour [1], the men's professional tennis league featuring the world's top 150 players, has developed a similar shot quality scale ranging from 0 to 10, applied to each shot type—forehand, backhand, serve, and return—as shown in Figure 1. Since each shot type has unique qualities like spin, depth, width, and speed that define its quality, their AI model scores each shot individually.



Figure 1: Example of ATP Tour shot quality player card [1].

Moses et al [2] employed three machine learning models to evaluate basketball shot success—determining whether a basketball shot would score—using numerical parameters such as the player's x and y positions and shot's air time. The models included a neural network with two layers of 20 nodes each, a binary logistic regression, and a gradient boosting model based on decision trees. The models achieved accuracies of 65%, 64.9%, and 65.1%, with corresponding F1 scores of 0.65, 0.72, and 0.71, based on a dataset of 84,466 shots. The gradient boosting model achieved the highest accuracy and second highest F1 score, although the differences among the models were minimal.

Kambhamettu et al. [3] employed multiple deep learning architectures—vanilla Recurrent Neural Networks (RNN), Long Short Term Memory (LSTM), and Gated Recurrent Units (GRU)—to classify the likelihood of a basketball shot being made or missed, using classification probabilities to quantify shot quality. They achieved an impressive 96.3% accuracy for backcourt shots, demonstrating the potential of deep learning architecture in sports analysis

METHODS

Ball Tracking using TrackNet CNN:

TrackNet, developed by Huang et. al [4] is a heatmap-based deep learning network for tracking high-speed and tiny objects. TrackNet's second network (TrackNet II) was employed. It enhances moving object detection by learning the ball's trajectory from three consecutive frames, unlike TrackNet I, which only uses a single input frame. TrackNet utilizes the heatmap-based CNN, which has proved useful for Human Pose Estimation [5] [6]. For each frame in a video, TrackNet predicts the pixel coordinate for the ball in the (X,Y) format for that frame. The first 13 layers of the CNN refer to VCG-16 [7], which is used for object classification. The 14-24 layers refer to DeconvNet [8], a de-convolutional neural network used for semantic segmentation, as shown in Figure 2. TrackNet's Model II achieved a precision of 99.8% and a Recall of 96.6%, making it a perfect choice for tennis ball tracking.

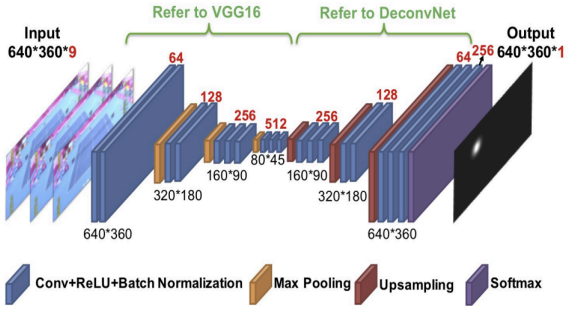


Figure 2: TrackNet model's architecture. [4]

Court Detection using Modified TrackNet CNN:

A modified version of the heatmap-based TrackNet CNN was utilized to detect the tennis court given an image, developed by Kosolapov [9]. In most videos, camera position is not stationary, so it's important to have court detection. The restructured model consists of an input tensor with just image, instead of three in TrackNet, and an output tensor that has 15 channels, 14 key points on the tennis court - "top-left doubles line", "top-right doubles line", "bottom-left doubles line", "bottom-right doubles line", "top-left singles line", "top-right singles line", "bottom-left singles line", "bottom-right singles line", "top-left service box", "top-right service box", "bottom-left service box", "bottom-right service box", "top-middle service box", "bottom-middle service box" - and one additional point which is the center of the vertical axis of the tennis court. These key points can be visualized in Figure 3. A homography matrix is then calculated based on these predicted key points in every frame in order to translate these points onto a reference court map, as shown in Figure 4. This allows us to track the ball and player relative to the court. Eventually, this perspective transform is implemented onto the players' (X,Y) pixel coordinates and the ball's (X,Y) pixel coordinates so everything is standardized on a 2D reference plane. The dimensions of the standardized court are 1665 (Width) units by 3496 (Height) units, where (0,0) is the top left corner of the reference plane. The court's coordinates are defined with the top left corner at (286, 561), top right at (1379, 561), bottom left at (286, 2935), and bottom right at (1379, 2935). Each translated unit is approximately 0.033 ft. Figure 5 shows us a simple representation of the perspective transform.

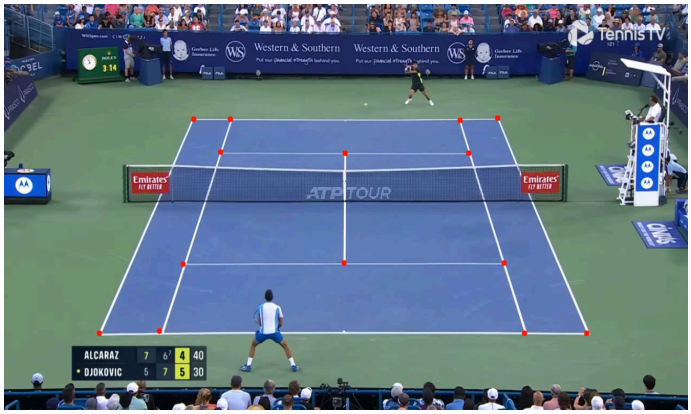


Figure 3: 14 key points are represented by red markers through modified TrackNet CNN for court detection

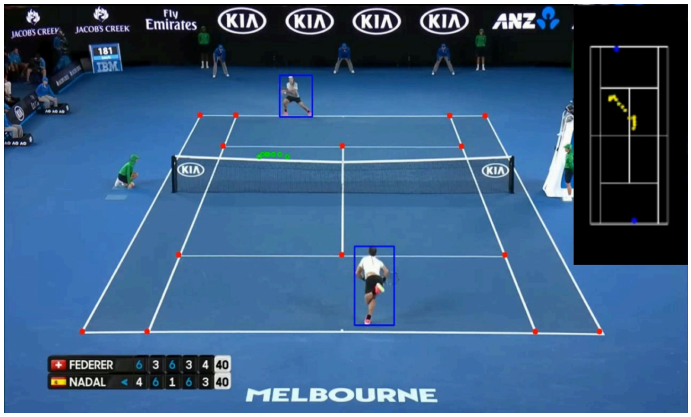


Figure 4: The translated coordinates can be visualized on a 2D reference plane in the top-right of the frame, where the yellow dots mark the ball, and the blue dots mark the players.

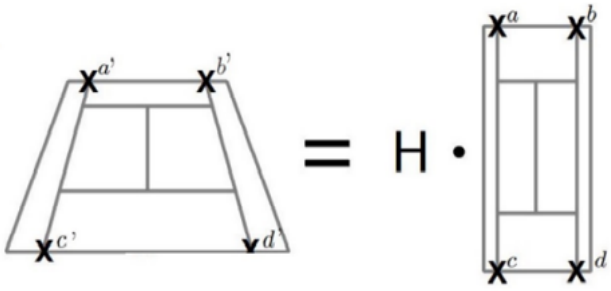


Figure 5: Visual showing what homography matrix does [9]

Bounce Detection using CatBoost Regressor:

CatBoost, developed by Prokhorenkova et. al [10] is an open-source machine learning library that is particularly effective for handling categorical features without requiring extensive preprocessing. Kosolapov [9] utilized the CatBoostRegressor model to predict when the tennis ball would bounce. The difference between x and y coordinates of neighboring ball coordinates was the main parameter that he used to train the model. Initially, mathematical patterns tried to be identified in bounce data, as shown in Figure 6, such as spikes in the derivatives of the ball's coordinates, to predict when it would bounce. Bounces can be seen in the figure as rapid jumps. However, after limited success, I opted to use Kosolapov's [9] pretrained model instead, which was trained using a small dataset consisting of 518 labeled bounce points, resulting in a precision of 74.4% and a recall of 73.2%.

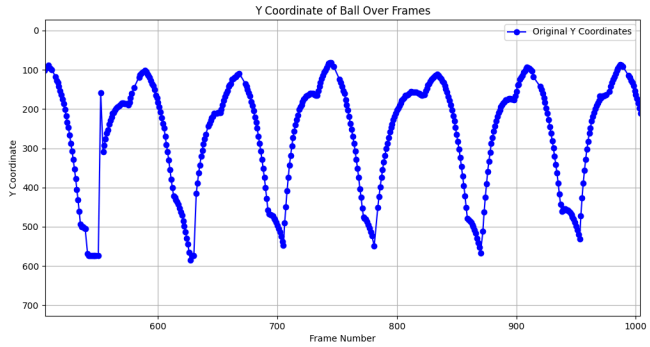


Figure 6: Graph of Y Coordinate of Ball over frames.

Player Detection using Faster R-CNN:

Faster R-CNN, developed by Ren et al. [11], is a powerful object detection framework trained on the COCO dataset, utilizing a Region Proposal Network (RPN) to generate accurate bounding boxes around detected objects. In sports applications, this model is employed for player detection, enabling precise tracking of player movements on the court. The ResNet-50 backbone, introduced by He et al. [12], enhances the model's feature extraction capabilities, improving detection accuracy and speed. Given an image, the bounding box is a prediction in the form [(array([x_min, y_min, x_max, y_max], dtype=float32), [x_center, y_center])], where "x_min, y_min" represent the coordinate for the top-left corner of the bounding box and "x_max, y_max" represent the bottom-right corner of the player's bounding box. The second element contains the x, y coordinate of the player. Equation 1 shows how the person point coordinate is calculated.

$$Person\ Point = \left(\frac{x_{\min} + x_{\max}}{2},\ y_{\max} \right)$$

Equation 1

Shot Quality Evaluation using Light GBM Model:

The Light Gradient-Boosting Machine developed by Ke et. al [13] and others at Microsoft is an extremely efficient implementation of the Gradient Boosting Decision Tree (GBDT) algorithm. It was designed mainly to address performance flaws in other Gradient Boosting models like XGBoost when dealing with high-dimensional data and large datasets. Light GBM's Gradient-based One-Side Sampling (GOSS) and Exclusive Feature bundling (EFB) allow it to reduce computational complexity by focusing on data points that contribute the most to the model's performance. Ke et. al [13] found that LightGBM speeds up the training process of a conventional GBDT by up to over 20 times while achieving almost the same accuracy. This optimization allowed for the model to perform well even with a relatively small dataset. Furthermore, unlike other boosting algorithms like XGBoost, Light GBM splits its tree leaf wise with the best fit rather than depth wise or level wise. Growing on the same leaf allows Light GBM to reduce more loss and hence results in much better accuracy than other decision tree based boosting algorithms.

DATASET

A general overview of how the numerical dataset was formed is shown in Figure 7.

Highlight videos from several professional ATP matches were downloaded through a command line tool named yt-dlp and were cut to just include footage of tennis points using Clipchamp, Microsoft's video editor. They were extracted at a resolution of 720p with a frame rate of 30 frames per second. To create the dataset, Sergey Kosolapov's GitHub repository, "Tennis Project" [14] was leveraged. It integrates the four primary models shown in Figure 7. The code was adapted to extract the specific numerical data required for the dataset.

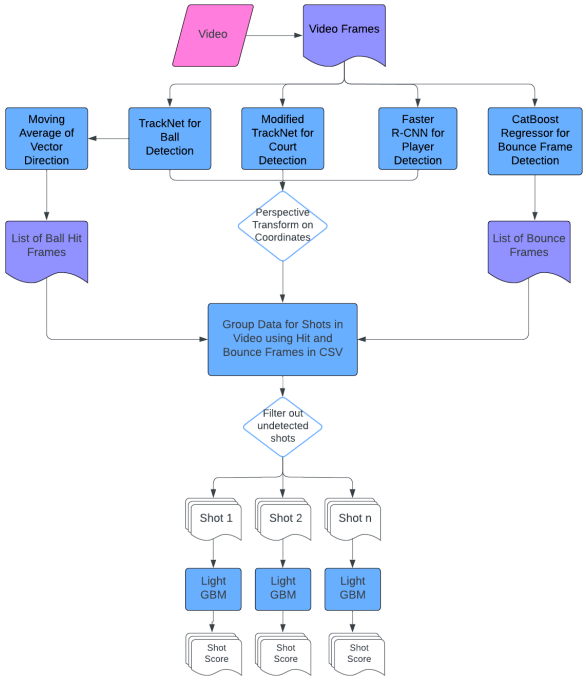


Figure 7: Overview of Dataset Creation

The TrackNet CNN is used to extract the ball's coordinates at each frame in the video. The ball's coordinates are saved in a tuple in the (X,Y) format: X represents the ball's pixel coordinate from 0 to 1080, and Y represents the ball's pixel coordinate from 0 to 720. If the ball is not detected for a said frame, then it is saved as (None, None) in the ball coordinate list. These undetected positions create gaps that must be addressed as part of the preprocessing steps prior to hit detection.

Hit Detection Data Collection:

To address this, linear interpolation was utilized to estimate the missing ball positions based on the known positions of the ball before and after missing segments of data. This method of imputation is reasonable because missing segments of data are usually not greater than a few frames. It is similar to using a tangent line at a known point to predict missing coordinates. If there are k missing frames between the start and end points, the interpolated position P_j (x_j, y_j) for each missing frame j (where $1 \leq j \leq k$) can be calculated using Equation 2 and Equation 3 for the ball's x and y positions, respectively. The term $(\frac{j}{k+1})$ represents the proportion of total distance between P_{start} and P_{end} that needs to be covered by the j-th frame.

P_{start} (x_{start}, y_{start}): The ball's position before the missing segment of frames
P_{end} (x_{end}, y_{end}): The ball's position after the missing segment

$$x_j = x_{start} + \left(\frac{j}{k+1}\right) \cdot (x_{end} - start)$$

Equation 2

$$y_j = y_{start} + \left(\frac{j}{k+1}\right) \cdot (y_{end} - y_{start})$$

Equation 3

The change in the y-coordinates is then tracked to determine the ball's vertical movement between consecutive frames, as shown by Equation 4, where $y_{current}$ and $y_{previous}$ are the y-coordinates of the ball in the current and previous frames.

$$y_{change} = y_{current} - y_{previous}$$

Equation 4

A positive y_{change} indicates that the ball is moving down the screen, while a negative y_{change} indicates upward movement across the screen. A binary representation of the ball's movement at each frame, as shown by Equation 5, is saved in a **directions** list when y_{change} has a magnitude other than 0. Frame numbers under these conditions are also recorded under a **frame_numbers** list.

$$\text{directions}(i) = \begin{cases} 1 & \text{if } y_{\text{change}}(i) > 0 \\ 0 & \text{if } y_{\text{change}}(i) < 0 \\ \text{excluded} & \text{if } y_{\text{change}}(i) = 0 \end{cases}$$

Equation 5

After preprocessing the data, a binary moving average with a window size of 15 frames is implemented on the **directions** list to analyze the ball's vertical movement. The binary moving average is computed using Equation 6 shown below. For each frame i, the binary direction of the previous seven and following seven frames (represented by j) is summed and divided by 15 to receive the binary moving average for the frame, which is added to a **binary_moving_avg** list. Equation 7 is the equation specifically used for a window size of 15. The moving average value cannot be computed for the first and last 7 values of the **directions** list, as doing so would result in accessing indexes that exceed the bounds of the directions list, so a **binary_moving_avg_frames** list is used to store frame numbers where the binary moving average is successfully calculated (Equation 8; w is window size).

$$binary_moving_avg[i] = \frac{1}{window_size} \cdot \sum_{j=0}^{window_size-1} directions\left[i - \frac{window_size - 1}{2} + j\right]$$

Equation 6

$$binary_moving_avg[i] = \frac{1}{15} \sum_{j=0}^{14} directions[i - 7 + j]$$

Equation 7

$$binary_moving_avg_frames = frame_numbers\left[\frac{w-1}{2} : -\frac{(w-1)}{2}\right]$$

Equation 8

A crossing at a said frame is detected if the binary moving average value transitions from less than 0.5 to greater than or equal to 0.5 or from greater or equal to 0.5 to less than 0.5 at that frame. This transition signifies a shot because a ball changes directions when hit. In other words, a shot occurs when the average vector direction of the ball over the seven frames preceding and following a certain frame crosses 0.5. Thus, frames for which this transition occurs are added to a **hits** list. An example of a crossings list for an input video might look like this: [40, 78, 120, 155], indicating that four shots were detected, with each element corresponding to the frame number where the hit occurred. Figure 8 shows a visual representation of the binary moving average and its crossings at 0.5. While no formal evaluation of the accuracy of this mathematical hit detection model was performed, qualitative observations confirm that it successfully tracked close to 90-95% of shots correctly in a video, with a margin of error of 2-3 frames.

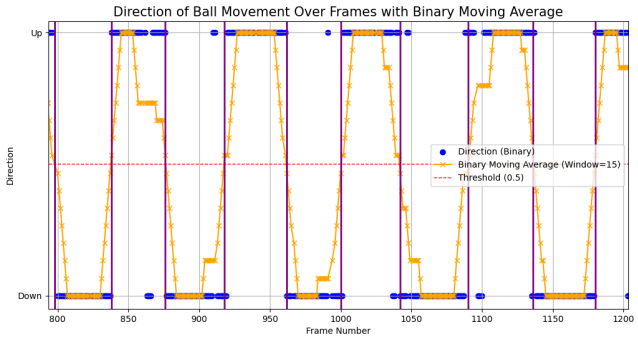


Figure 8: Visual representation of binary moving average, where vertical purple line represents frames where crossings occurred.

Court Detection Improvement For Data Collection:

In order to diversify the types of courts and camera angles the court detector model could support, the court detection model was also successfully retrained using 1000 additional frames, with 800 allocated for training and 200 for testing. 250 images were drawn across 5 distinct matches with untraditional camera angles in comparison to professional highlight videos. From each set, 200 frames were used for training and 50 for testing. I chose videos without camera movement, so each set of 200 frames only needed to be labeled once with 14 key points. In total, the training dataset consisted of 10,098 frames. This improved the court detection model and was then used in the remainder of the project. The improvement can be seen in a before-after comparison in Figures 9 and 10.



Figure 9: Test Image in frame before improved Court Detection Model



Figure 10: Test Image in same frame after improved Court Detection Model

Player Data Collection:

Each player's pixel coordinates are saved in a **persons_bottom** and **persons_top** list; Each element in these lists is stored as its own list, containing an array with the player's bounding box information—[Xmin, Ymin, Xmax, Ymax] in float32 format—along with a tuple that holds the localized center coordinates [Xcenter, Ycenter]. Before performing the perspective transformation on both players' pixel coordinates and adding the top and bottom players' coordinates to a single **person_list**, several checks are necessary to ensure that incorrect data isn't included. As shown in Figure 11, extraneous individuals had to be filtered out when the length of any element in either the persons_bottom or persons_top directory exceeded 1. Additionally, if no person is detected in either player directory (top or bottom) for a frame, the tuple (None, None) is set to the missing element to keep the length of both player lists uniform with the number of frames in the inputted video. The perspective transformation is then applied to the localized coordinates [Xcenter, Ycenter] for both players in each frame. These coordinates are subsequently added as tuples in the format (X, Y), with the top player's data first, followed by the bottom player's, to the **person_list**.

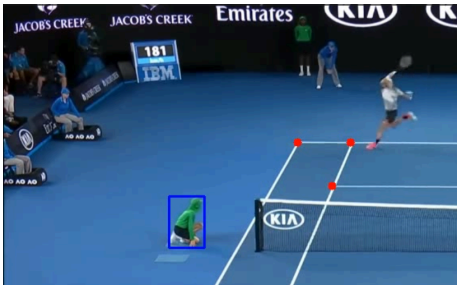


Figure 11: Example of frame where a ball-boy is being detected in addition to the actual player. It is important to note that a bounding box is only displayed around the individual with the highest probability of being a player. However, both players' data is still tracked.

Bounce Data Collection:

The pretrained CatBoost Regressor model is used to extract the frames where it predicts the ball bounces and saves those frames in a set called **bounces**. Example: {36, 78, 110, 140} - the individual elements in the set represent frame numbers.

Assembling Data Structure:

After the **hits** list (the list of frames where the ball is hit), the **bounces** set (set of frames where ball bounces), the transformed **person_list** (list of person coordinates), and transformed **ball_coordinates** list (list of ball coordinates) are correctly saved for an inputted video, the tangible data structure can be formed. A tennis shot is classified by its hit and subsequently its bounce, so a function is used to pair each frame in the hits list to its correlated bounce in the bounces set. The output is a **hit_bounce_pairs** list where each element is a tuple formatted as (hit frame, bounce frame). Cases where untracked bounces or hits occur are taken care of in the function.

Following the division of the **person_list** into **person_top_list** and **person_bottom_list**, the necessary features can be accessed for the data structure. Each shot in the data frame includes the parameters shown in Table 1. The features corresponding to the correct player are selected based on the Y coordinate of the bounce. For instance, when the bounce occurs on the bottom side of the court, it can be inferred that the player positioned at the bottom is the receiver, while the player at the top is the hitter. Consequently, their respective data can be extracted from the correct player directory.

Feature	**Description of Feature**
Hit Frame	Frame ball was hit
Bounce Frame	Frame ball bounces
Ball Position at Bounce	(X,Y) coordinates of bounce
Receiver's Player Position at Bounce	Person receiving the ball's (X,Y) coordinates at bounce
Hitter's Player Position at Ball Impact	Person hitting's (X,Y) coordinate when they hit the ball
Receiver's Player Position at Ball Impact	Person receiving's (X,Y) coordinate when the ball is hit

Table 1: Features that are used to train the Light GBM Model.

Experts, including Loffing et. al [15], have shown that key factors for successful shots are its depth into the court, its horizontal positioning, its distance from the receiving player, its speed, and both players' positioning. All of these factors are either directly or indirectly tracked by the features in Table 1 with the exception of the ball's velocity. Tracking the ball's average velocity for each shot was attempted, but results could not be verified and were sporadic. Thus, it was not included as a feature. Each shot is labeled using the key factors highlighted in Table 1 on a scale between 0 and 10 (shown in Column 7 of Table 2). It is important to note that the labeling is conducted by an individual with tennis expertise, so inherently it is subjective. Table 2 visualizes the data set by shot (rows) and pertinent coordinate data in the columns for the first few shots in a video. Before training the model, each parameter was split up into its individual X and Y components.

Hit Frame	Bounce Frame	Ball Position at Bounce	Player Position of Receiver at Bounce	Player Position of Person Hitting When Ball Is Hit	Player Position of Receiver When Ball Is Hit	Shot Score
12	42	(723, 1071)	(850, 542)	(605, 3292)	(964, 623)	5
58	76	(575, 2607)	(800, 3441)	(834, 592)	(883, 3395)	6
110	155	(1007, 2362)	(831, 3194)	(993, 590)	(524, 3446)	6
168	189	(627, 1227)	(712, 362)	(848, 3057)	(1014, 409)	5
246	288	(744, 940)	(878, 701)	(709, 3306)	(773, 446)	6
301	324	(496, 2767)	(680, 3409)	(890, 688)	(831, 3346)	8
358	380	(1128, 2285)	(457, 3188)	(647, 1195)	(369, 3399)	7

Table 2: A visualization of the dataset after labeling

If any parameter contains None values, the corresponding values from the next frame where the parameter is not None will be used. Missing segments of data are usually never greater than 4-5 frames, so this fix is reasonable. Additionally, numerous shots in a video could not be tracked due to issues such as missed bounces, inaccurate bounce detection, and occasional errors in the hit detection model. Approximately 20-30% of shots per video, on average, are not tracked.

Training the LightGBM Model:

With the dataset composed of features extracted from the TrackNet CNN, CatBoostRegressor, and Faster R-CNN, the LightGBM model was trained on 208 labeled shots using the parameters outlined in Table 3, with an 80-20 train-test split. The x-axis shows the number of boosting iterations, denoted by N_estimators, and the y-axis represents the loss, measured as mean squared error. The validation loss, as shown in Figure 12, showed

a minimal increase between approximately 40 and 1000 boosting iterations, reaching its lowest point around 100 iterations. Both validation and training loss were measured by mean squared error, representing the average squared deviation from the assigned shot score on a scale from 0 to 10.

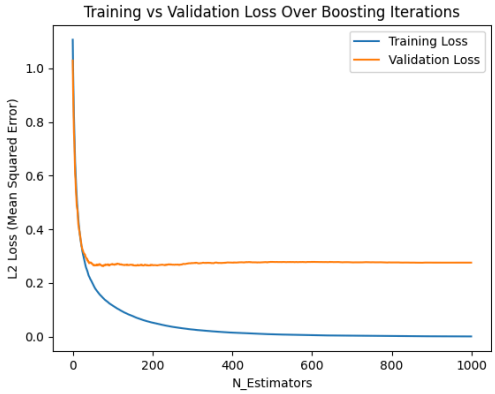


Figure 12: The loss curve of LightGBM model training.

Parameter	**Value**
n_estimators	100
Objective	'regression'
Learning Rate	0.1
Max Depth	-1
Feature Fraction	1
Bagging Fraction	1
Random Seed	42
Loss Metric	'mean squared error'

Table 3: Parameters used to train LightGBM Model

RESULTS AND DISCUSSION

The Light GBM model recorded a mean squared error of 0.27, a mean absolute percentage error of 7.31%, and a mean absolute error of 0.44 using only 208 data points to evaluate shot scores in the testing dataset. In contrast, the standard GBM model achieved a mean squared error of 0.22 with the same parameters detailed in Table 2. This difference may be due to LightGBM's sensitivity to data characteristics and its need for specific hyperparameter tuning. Table 4 displays the relative importance each feature had in determining the quality of a shot. The model notably aligns with the common belief among tennis players that the ball's vertical and lateral positions on the court are key factors in determining its strength, as "Feature Ball Position X" and "Feature Ball Position Y" contributed the most to the model's evaluation, with importance values of 22.08% and 22.63%, respectively. By attaining relatively low MSE values, my objective to assess shot quality through the extraction of numerical data from necessary models was fulfilled. However, there are many improvements that can and should be made to this project in the future.

Feature Type	**Feature Importance**
Feature Ball Position X	22.08%
Feature Ball Position Y	22.63%
Feature Player Receiver X	6.75%
Feature Player Receiver Y	7.85%
Feature Player Hitting X	10.85%
Feature Player Hitting Y	6.57%
Feature Player Receiver When Hit X	9.85%
Feature Player Receiver When Hit Y	13.69%

Table 4: Relative Feature Importance for Light GBM Model

Similarly, the heatmap in Figure 13 illustrates that shots targeted at the wider and deeper regions of the court yield higher scores, while those aimed toward the center and shallower areas tend to score lower. This figure correlates the X and Y positions of the ball with its shot score, where the x-axis represents the length of the tennis court and the y-axis spans from the top to the bottom of the court. In Figure 14, which provides a 3D perspective of Figure 13, it is clear that shot scores decrease as shots

approach the horizontal center of the court (represented by the x-axis) and move vertically toward the net (represented by the y-axis). This pattern resembles a slanted absolute value function, with its peak at the center of the court, or the net, within the x/y plane of Figure 14. With additional data points of balls bouncing closer to the center of the court, the trend will become more evident.

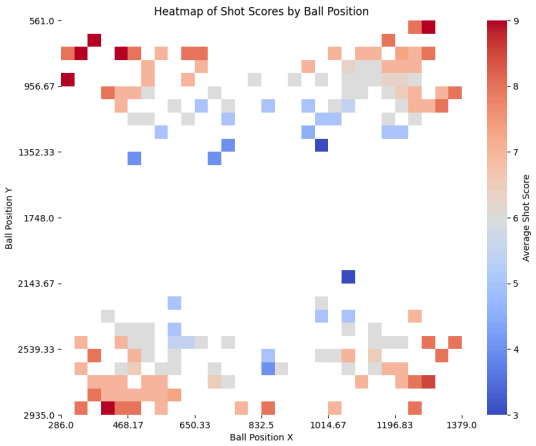


Figure 13: Heatmap of shot scores by ball position

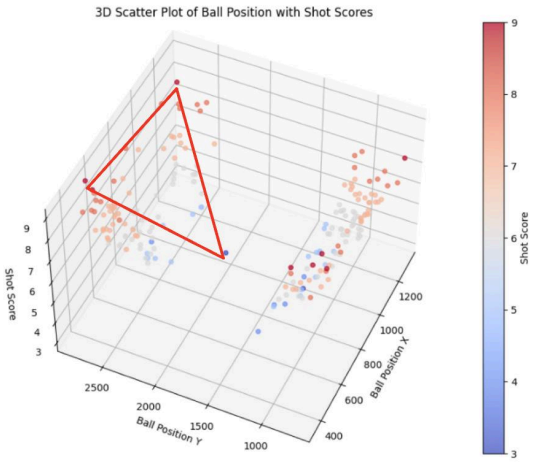


Figure 14: 3D representation of Ball Position X/Y with Shot Scores

One important limitation that this model has is that it does not support doubles match play, where two people are on both sides of the court. Doubles is a key aspect of tennis, introducing unique dynamics that differ from singles in terms of strategy, positioning, and movement coordination. To enable this model to effectively work for doubles matchplay, the key requirements include implementing accurate person detection for all four players, expanding the dataset, and adjusting the model's parameters to account for the different dynamics in doubles. Another key limitation of this model is its inability to track volleys—shots that are hit before bouncing on the ground. Since the model classifies shots based on their hit and bounce, it cannot analyze shots whose bounces are not tracked.

One potential source of error comes from the fact that the data was labeled by just one individual and was trained with a relatively small dataset of 208 data points. The model could benefit from greater diversity if other coaches or high-level tennis players contributed to labeling shots. This would expand the dataset and include a broader range of perspectives, as attaining a perfect ground truth is near impossible in this case due to the subjective nature of shot scoring. Additionally, shots at the extreme ends of the quality scale are relatively uncommon in matches; as a result, when they do occur, the model tends to skew their assessment due to its inherent bias toward more frequently observed scores. To improve accuracy, targeted labeling of these extreme shots would be necessary.

The final output gathers all the necessary data described in the dataset section for correctly tracked shots in an inputted video and displays the predicted shot score to the ball's pixel coordinates (un-translated) from its hit to its bounce and returns a video output using OpenCV, as shown in Figures 15 and 16.A visual representation of Light GBM's predictions can be viewed in video:
DjokovicZverevShotScore.mp4

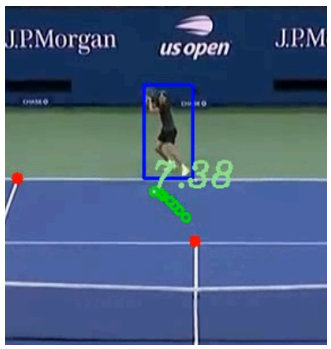


Figure 15: Shot score displayed next to ball on way up.

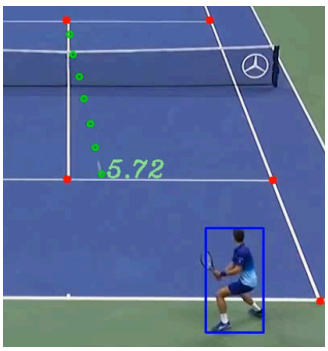


Figure 16: Shot score displayed next to ball on way down.

CONCLUSION

Overall, the proposed integration of the bounce detection model (CatBoostRegressor), mathematical

moving average hit detection model, player detection model (Faster R-CNN), ball detection model (TrackNet CNN), and court detection model (Modified TrackNet CNN), all aimed at gathering numerical data to train the LightGBM model, demonstrated amazing potential for assessing shot quality. It achieved a good mean squared error of 0.27 when classifying shots. The accuracy of the model will most likely be improved by a larger dataset and diversifying the individuals labeling for the model.

Further investigation into improvements for the model includes improving the dataset for the Light GBM model to be trained with, like adding more types of shots that don't occur too often. Another important improvement would include improving the CatBoostRegressor model for ball detection, which achieved a precision of 74.4% and a recall of 73.2%. As a result, many bounces went untracked in video and many shots could not thus be classified. Potentially proposing a different machine learning model or tweaking the CatBoostRegressor's parameters to attempt to increase its accuracies would drastically improve the model in terms of its practicality for users. The pretrained CatBoostRegressor model was also trained on a limited dataset of 518 data points, which may have further contributed to the model's suboptimal performance. It would also be intriguing to compare this numerical model with a CNN for assessing shot quality, alongside other machine learning models like those used by Kambhamettu et al. [3], who showcased the potential of diverse machine learning approaches in sports analysis.

This research could be integrated into existing apps like Swing Vision, a tennis analytics platform that enables users to upload videos and receive detailed match analytics. Swing Vision already provides highly accurate bounce and court detection, as illustrated in Figure 17. While the backend of their technology is proprietary, incorporating a shot score feature could enhance the app's capabilities.

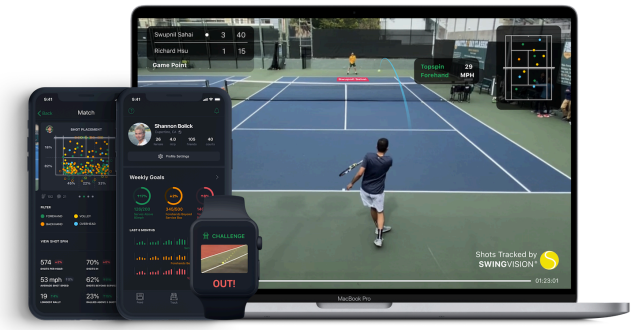


Figure 17: Display of Swing Vision's accurate court and ball detection [16]

ACKNOWLEDGEMENTS

I would like to thank Mr. Andrei Isichenko and the Inspirit AI team for guiding me through this project.

REFERENCES

[1] ATP Tour, "Insights: Shot quality," [Online]. Available: https://www.atptour.com/en/news/insights-shot-quality.

[2] Moses, M. Analysis of Machine Learning Models Predicting Basketball Shot Success. 2020 International Young Researchers' Conference, pp. 124-128.

[3] Kambhamettu, A.; Shrivastava, A.; Gwilliam, M. Quantifying NBA Shot Quality: A Deep Network Approach. MM Sports '24: Proceedings of the 7th ACM International Workshop on Multimedia Content Analysis in Sports, pp. 91-95.

[4] Huang, Y.-C.; Liao, I.-N.; Chen, C.; Ik, T.; Peng, W. TrackNet: A Deep Learning Network for Tracking High-speed and Tiny Objects in Sports Applications. 16th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), 2019, pp. 18-21.

[5] Belagiannis, V.; Zisserman, A. Recurrent human pose estimation. 2017 12th IEEE International Conference on Automatic Face and Gesture Recognition (FG 2017). IEEE, 2017, pp. 468-475.

[6] Pfister, T.; Charles, J.; Zisserman, A. Flowing convnets for human pose estimation in videos. IEEE International Conference on Computer Vision, 2015, pp. 1913-1921.

[7] Simonyan, K.; Zisserman, A. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556, 2014.

[8] Noh, H.; Hong, S.; Han, B. Learning deconvolution network for semantic segmentation. IEEE International Conference on Computer Vision, 2015, pp. 1520– 1528

[9] Sergey, K. Tennis analysis using deep learning and machine learning. Google Research, https://medium.com/@kosolapov.aetp/tennis-analysis-using-deep-learning-and-machine-learning-a5a74db7e2ee

[10] Prokhorenkova, L.; Gusev, G.; Vorobev, A.; Dorogush, A.; Gulin, A. (2018b). CatBoost: unbiased boosting with categorical features. arXiv: 1706.09516, 2017.

[11] Ren, S.; He, K.; Girshick, R.; Sun, J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. arXiv:1506.01497, 2015.

[12] He, K.; Zhang, X.; Ren, S.; & Sun, J. Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770-778.

[13] Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.-Y. Light GBM: A Highly Efficient Gradient Boosting Decision Tree. NIPS' 17: Proceedings of the 31st International Conference on Neural Information Processing Systems, pp. 3149-3157.

[14] Kosolapov, S. "TennisProject," GitHub, 2023. [Online]. Available: https://github.com/yastrebksv/TennisProject.

[15] Loffing, F.; Hagemann, N. On-Court Position Influences Skilled Tennis Players' Anticipation of Shot Outcome. Journal of Sport and Exercise Psychology, 36 (1), 2014, pp. 14-26.

[16] Chua, J. "Startup Spotlight: SwingVision." Sports Technology Blog, 20 Dec. 2021, https://sportstechnologyblog.com/2021/12/20/startup-spotlight-swingvision/.